Web and Store Scaling Risks

Overview

The browser surfaces are functional, but they are implemented using two very dif
ferent architectures: a React SPA for `/web` and a multipage static storefront f
or `/store`. This split is workable, but it introduces duplication and operation
al drag.

Main risks

1. split frontend architecture

- `/web` uses React, hooks, and route-based state
- `/store` uses static HTML and page-specific browser scripts
- shared behavior such as auth, notifications, and role gating is duplicated in
different styles

2. proxy-heavy store model

- the store layer proxies many endpoints to another API base
- this increases request latency, observability complexity, and failure modes

3. platform drift

- Android, web SPA, and store all implement overlapping product concepts differe
ntly
- notification routing, role checks, feed behavior, and localization can diverge

4. browser-local state concentration

- the web app and the store both lean heavily on local storage
- auth/session assumptions are spread across utilities and page scripts

5. mixed deployment surfaces

- `/web` is built output served from backend public assets
- `/store` is static multipage content plus a route/proxy layer
- `/blog` is another separate static content surface

Recommended improvements

1. define shared frontend contracts for auth, notifications, roles, and preferre
d language
2. reduce store proxy breadth or isolate it into a clearer commerce service boun
dary
3. keep documentation and RAG ingestion segmented by client surface so retrieval
 stays precise